Reparto de tareas — PetCare: la app completa (MVVM + Room + Hilt + Navigation Compose + DataStore)

Contexto

Esta versión del plan deja de tratarse como "una entrega parcial" y pasa a cubrir **la app completa** que piden los dos documentos del profesor (Trabajo Grupal 1 — los 9 wireframes — y Trabajo Autónomo Etapa 2 — la arquitectura técnica), más las correcciones de la clase sobre Hilt/NavHost ya aplicadas en la versión anterior.

Sobre lo ya construido (Home, Mis Mascotas, Detalle, Registrar/Editar Cuidado, con Hilt + NavHost + Room de 3 entidades) se agrega lo que faltaba para que sea la app completa:

1. **Splash Screen** (wireframe "Pantalla 1").
2. **CRUD completo de** `Mascota` (RF01–RF04: ya no es de solo lectura) — pantalla `RegistrarMascotaScreen` (crear + editar, mismo patrón que `RegistrarCuidadoScreen`) y botón "Eliminar" en Detalle.
3. **Pantalla de Configuración con DataStore** (RF10, wireframe "Pantalla 9") — tema claro/oscuro, orden de mascotas, orden del historial.
4. **Navegación inferior (bottom bar)** con 3 pestañas: Inicio, Mascotas, Ajustes — así se llega a Configuración, y es el diseño que ya traían los wireframes de alta fidelidad.

Las pantallas "Historial de cuidados" y "Detalle del cuidado" del wireframe siguen fusionadas (dentro de Detalle de mascota y de `RegistrarCuidadoScreen`, respectivamente), como ya se había decidido.

Supuestos de esta versión (avísame si alguno no es lo que quieres)

- **Bottom bar con 3 pestañas, no 4**: los wireframes muestran un cuarto ícono "Cuidados" en la barra inferior, pero como un listado global de cuidados de *todas* las mascotas todavía no se definió en ningún requisito funcional (RF05-RF08 hablan de cuidados por mascota, accedidos desde su Detalle), se deja fuera para no inventar una pantalla no especificada. Los cuidados se siguen viendo desde Detalle de cada mascota.
- **Las preferencias de Configuración se guardan y se leen correctamente con DataStore** (que es literalmente lo que pide RF10), pero **aplicarlas visualmente en el resto de la app** (cambiar el tema global, reordenar las listas de Mascotas/Historial) se deja como mejora opcional — no es requisito explícito, y evita reabrir el trabajo ya terminado de Personas B y C.
- `Mascota` al eliminarse borra en cascada sus `Cuidado` asociados (ya estaba configurado así con `ForeignKey.CASCADE`).

Cómo esto encaja con la rúbrica de la Etapa 2

Criterio de la rúbrica	¿Se cubre?
1. Capa de UI & State Hoisting (Compose, Material 3)	Sí, en las 7 pantallas — `Route`/`Screen`, `hiltViewModel()`, `collectAsStateWithLifecycle()`.

Criterio de la rúbrica	¿Se cubre?
2. Arquitectura MVVM & Gestión de Estado Reactivo	Sí — ViewModel por pantalla, `StateFlow<UiState>` sellado, eventos puntuales con `Channel` en los 2 formularios y en la eliminación de mascota.
3. Capa de Datos (Room & Single Source of Truth)	Sí — 3 entidades, CRUD completo en `Mascota` y `Cuidado`.
4. Patrón Repositorio y Desacoplamiento	Sí — `MascotaRepository`, `CuidadoRepository`, `PreferenciasRepository`, todos con implementación `Offline*`.
5. Navegación e Inyección de Dependencias (DI)	Sí — NavHost con 7 rutas tipadas + bottom bar, Hilt para toda la inyección.

Estructura de paquetes — app completa

```
app/src/main/java/com/am/septimob/petcare/
■■■ PetCareApplication.kt              (@HiltAndroidApp)
■■■ PetCareRoute.kt                    (7 rutas tipadas)
■■■ PetCareApp.kt                      (Persona D — NavHost + bottom bar)
■■■ data/
■   ■■■ Mascota.kt                     (@Entity — tabla maestra 1)
■   ■■■ TipoCuidado.kt                 (@Entity — tabla maestra 2)
■   ■■■ Cuidado.kt                     (@Entity — tabla transaccional)
■   ■■■ MascotaDao.kt                  (CRUD completo — AMPLIADO)
■   ■■■ TipoCuidadoDao.kt
■   ■■■ CuidadoDao.kt                  (CRUD completo)
■   ■■■ PetCareDatabase.kt
■   ■■■ MascotaRepository.kt + OfflineMascotaRepository.kt     (AMPLIADO)
■   ■■■ CuidadoRepository.kt + OfflineCuidadoRepository.kt
■   ■■■ UserPreferences.kt             (NUEVO — data class + enums Tema/OrdenMascotas/OrdenHistorial)
■   ■■■ PreferenciasRepository.kt + OfflinePreferenciasRepository.kt   (NUEVO, usa DataStore)
■■■ di/
■   ■■■ DatabaseModule.kt
■   ■■■ RepositoryModule.kt            (AMPLIADO — ahora liga también PreferenciasRepository)
■   ■■■ DataStoreModule.kt             (NUEVO)
■■■ SplashScreen.kt                                          (Persona A — NUEVO)
■■■ HomeScreen.kt + HomeViewModel.kt                         (Persona A)
■■■ MascotasScreen.kt + MascotasViewModel.kt                 (Persona B)
■■■ RegistrarMascotaScreen.kt + RegistrarMascotaViewModel.kt (Persona B — NUEVO)
■■■ DetalleMascotaScreen.kt + DetalleMascotaViewModel.kt     (Persona C — AMPLIADO: editar/eliminar
■■■ RegistrarCuidadoScreen.kt + RegistrarCuidadoViewModel.kt (Persona C)
■■■ ConfiguracionScreen.kt + ConfiguracionViewModel.kt       (Persona D — NUEVO)
```

Reparto final (extendiendo los roles ya definidos)

Persona	Pantallas	Qué más le toca
A	Splash + Home	—

Persona	Pantallas	Qué más le toca
B	Mis Mascotas + Registrar/Editar Mascota (nueva)	Ampliar `MascotaDao`/`MascotaRepository` con `guardar`/`actualizar`/`eliminar`
C	Detalle (ahora con editar/eliminar mascota) + Registrar/Editar Cuidado	Agregar evento de "mascota eliminada" en `DetalleMascotaViewModel`
D	Configuración (nueva, con DataStore)	NavHost con 7 rutas + bottom bar, todos los módulos de Hilt, `PreferenciasRepository`

Aviso: la Tarea de Persona D creció bastante (navegación + bottom bar + 3 módulos de Hilt + su propia pantalla). Se recomienda que arranque primero y que quien termine rápido (probablemente Persona A, que tiene las 2 pantallas más simples) le ayude con el armado del `NavHost`/bottom bar.

Paso 0 — Base compartida

0.1 — Dependencias en Gradle (ahora con DataStore además de Room/Hilt/Navigation)

Sobre lo ya agregado (KSP, `kotlin.plugin.serialization`, Hilt, Room, Navigation Compose, `lifecycle-viewmodel-compose`, `hilt-navigation-compose`), agregar:

- `androidx.datastore:datastore-preferences`.

Usar siempre la versión estable más reciente compatible con el resto del proyecto — revisar en Android Studio.

0.2 — `PetCareRoute.kt` (ahora con 7 rutas)

```
package com.am.septimob.petcare

import kotlinx.serialization.Serializable

sealed interface PetCareRoute {
    @Serializable data object Splash : PetCareRoute
    @Serializable data object Home : PetCareRoute
    @Serializable data object Mascotas : PetCareRoute
    @Serializable data object Configuracion : PetCareRoute
    @Serializable data class Detalle(val mascotaId: Int) : PetCareRoute
    @Serializable data class RegistrarMascota(val mascotaId: Int? = null) : PetCareRoute
    @Serializable data class RegistrarCuidado(val mascotaId: Int, val cuidadoId: Int? = null) : PetCareR
}
```

0.3 — Contrato de datos (bloqueante)

Las entidades/interfaces de `Mascota`/`TipoCuidado`/`Cuidado` no cambian de forma (ver versión anterior del documento), salvo que `MascotaRepository` **crece**:

```
// data/MascotaRepository.kt
interface MascotaRepository {
    fun obtenerMascotas(): Flow<List<Mascota>>
    fun obtenerMascotaPorId(id: Int): Flow<Mascota?>
    suspend fun guardar(mascota: Mascota): Long
    suspend fun actualizar(mascota: Mascota)
    suspend fun eliminar(mascota: Mascota)
}
```

Y se agrega el contrato de preferencias (independiente de Room):

```
// data/UserPreferences.kt
enum class Tema { CLARO, OSCURO }
enum class OrdenMascotas { POR_NOMBRE, MAS_RECIENTE }
enum class OrdenHistorial { RECIENTES_PRIMERO, ANTIGUOS_PRIMERO }

data class UserPreferences(
    val tema: Tema = Tema.CLARO,
    val ordenMascotas: OrdenMascotas = OrdenMascotas.POR_NOMBRE,
    val ordenHistorial: OrdenHistorial = OrdenHistorial.RECIENTES_PRIMERO
)
```

```
// data/PreferenciasRepository.kt
interface PreferenciasRepository {
    val preferencias: Flow<UserPreferences>
    suspend fun actualizarTema(tema: Tema)
    suspend fun actualizarOrdenMascotas(orden: OrdenMascotas)
    suspend fun actualizarOrdenHistorial(orden: OrdenHistorial)
}
```

`CuidadoRepository` no cambia respecto a la versión anterior.

0.4 — Sub-hilo 1: implementación de Mascota (ahora con CRUD completo)

```
// data/MascotaDao.kt
@Dao
interface MascotaDao {
    @Query("SELECT * FROM mascotas")
    fun obtenerTodas(): Flow<List<Mascota>>

    @Query("SELECT * FROM mascotas WHERE id = :id")
    fun obtenerPorId(id: Int): Flow<Mascota?>

    @Insert
    suspend fun insertar(mascota: Mascota): Long

    @Update
    suspend fun actualizar(mascota: Mascota)

    @Delete
    suspend fun eliminar(mascota: Mascota)
}
```

```
// data/OfflineMascotaRepository.kt
class OfflineMascotaRepository @Inject constructor(
    private val dao: MascotaDao
) : MascotaRepository {
    override fun obtenerMascotas(): Flow<List<Mascota>> = dao.obtenerTodas()
    override fun obtenerMascotaPorId(id: Int): Flow<Mascota?> = dao.obtenerPorId(id)
    override suspend fun guardar(mascota: Mascota): Long = dao.insertar(mascota)
    override suspend fun actualizar(mascota: Mascota) = dao.actualizar(mascota)
    override suspend fun eliminar(mascota: Mascota) = dao.eliminar(mascota)
}
```

0.5 — Sub-hilo 2: implementación de TipoCuidado + Cuidado (sin cambios respecto a la versión anterior)

`TipoCuidadoDao`, `CuidadoDao` y `OfflineCuidadoRepository` quedan igual que en la versión anterior de este documento.

0.6 — Sub-hilo 3 (nuevo, independiente de Room): implementación de Preferencias con DataStore

```
// data/OfflinePreferenciasRepository.kt
class OfflinePreferenciasRepository @Inject constructor(
    private val dataStore: DataStore<Preferences>
) : PreferenciasRepository {

    private object Claves {
        val TEMA = stringPreferencesKey("tema")
        val ORDEN_MASCOTAS = stringPreferencesKey("orden_mascotas")
        val ORDEN_HISTORIAL = stringPreferencesKey("orden_historial")
    }

    override val preferencias: Flow<UserPreferences> = dataStore.data.map { prefs ->
        UserPreferences(
            tema = prefs[Claves.TEMA]?.let { Tema.valueOf(it) } ?: Tema.CLARO,
            ordenMascotas = prefs[Claves.ORDEN_MASCOTAS]?.let { OrdenMascotas.valueOf(it) } ?: OrdenMasc
            ordenHistorial = prefs[Claves.ORDEN_HISTORIAL]?.let { OrdenHistorial.valueOf(it) } ?: OrdenH
        )
    }

    override suspend fun actualizarTema(tema: Tema) {
        dataStore.edit { it[Claves.TEMA] = tema.name }
    }

    override suspend fun actualizarOrdenMascotas(orden: OrdenMascotas) {
        dataStore.edit { it[Claves.ORDEN_MASCOTAS] = orden.name }
    }

    override suspend fun actualizarOrdenHistorial(orden: OrdenHistorial) {
        dataStore.edit { it[Claves.ORDEN_HISTORIAL] = orden.name }
    }
}
```

0.7 — Unión: módulos de Hilt (la hace quien lidera Persona D)

```
// di/DataStoreModule.kt
private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "petcare_preferenci

@Module
@InstallIn(SingletonComponent::class)
object DataStoreModule {
    @Provides
    @Singleton
    fun provideDataStore(@ApplicationContext context: Context): DataStore<Preferences> = context.dataSto
}
```

```
// di/RepositoryModule.kt — ahora liga también Preferencias
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindMascotaRepository(impl: OfflineMascotaRepository): MascotaRepository

    @Binds
    @Singleton
    abstract fun bindCuidadoRepository(impl: OfflineCuidadoRepository): CuidadoRepository

    @Binds
```

```
    @Singleton
    abstract fun bindPreferenciasRepository(impl: OfflinePreferenciasRepository): PreferenciasRepository
}
```

`DatabaseModule.kt`, `PetCareApplication.kt` y `MainActivity.kt` quedan igual que en la versión anterior de este documento.

Paso 1 — Trabajo en paralelo

Regla estricta sin cambios: `XRoute` es el único que conoce el ViewModel (vía `hiltViewModel()` como valor por defecto de un parámetro); `XScreen` solo recibe `UiState` + lambdas. Material 3 en todas las pantallas.

Persona A — `SplashScreen.kt` (nueva) + `HomeScreen.kt`/`HomeViewModel.kt` (sin cambios)

```
// SplashScreen.kt
@Composable
fun SplashRoute(onTimeout: () -> Unit) {
    LaunchedEffect(Unit) {
        delay(1500)
        onTimeout()
    }
    SplashScreen()
}

@Composable
fun SplashScreen() {
    // UI estática según wireframe "Pantalla 1": logo/ícono de PetCare, título,
    // el eslogan "El cuidado de tus mascotas en un solo lugar", un indicador de
    // carga y el texto "Tus datos se guardan en este dispositivo".
}

@Preview
@Composable
private fun SplashScreenPreview() {
    SplashScreen()
}
```

No tiene ViewModel propio (no necesita leer nada de la base de datos), por eso no sigue el patrón `Route`/ViewModel de las demás — solo espera y navega.

`HomeScreen.kt`/`HomeViewModel.kt` quedan igual que en la versión anterior de este documento.

Persona B — `MascotasScreen.kt`/`MascotasViewModel.kt` (sin cambios) + `RegistrarMascotaScreen.kt`/`RegistrarMascotaViewModel.kt` (nueva)

`MascotasRoute` ahora también recibe `onAgregarMascotaClick: () -> Unit` para el botón "+" de agregar mascota (wireframe "Pantalla 3"), que el contenedor conecta a `PetCareRoute.RegistrarMascota()` (sin `mascotaId`, o sea modo crear).

```
sealed interface RegistrarMascotaUiState {
    data object Cargando : RegistrarMascotaUiState
    data class Formulario(
        val nombre: String = "",
        val especie: String = "",
        val raza: String = "",
        val fechaNacimiento: String = "",
        val sexo: String = "",
        val pesoKg: String = "",
        val esEdicion: Boolean = false
    ) : RegistrarMascotaUiState
    data class Error(val mensaje: String) : RegistrarMascotaUiState
```

```
}

sealed interface RegistrarMascotaEvento {
    data object GuardadoExitoso : RegistrarMascotaEvento
    data class Error(val mensaje: String) : RegistrarMascotaEvento
}

@HiltViewModel
class RegistrarMascotaViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val repository: MascotaRepository
) : ViewModel() {
    private val ruta = savedStateHandle.toRoute<PetCareRoute.RegistrarMascota>()

    private val _eventos = Channel<RegistrarMascotaEvento>()
    val eventos = _eventos.receiveAsFlow()

    // uiState: si ruta.mascotaId != null, carga esa mascota y arma Formulario(esEdicion = true);
    // si es null, arranca con un Formulario vacío (esEdicion = false).

    fun guardar(nombre: String, especie: String, raza: String, fechaNacimiento: String, sexo: String, pe
        viewModelScope.launch {
            try {
                val mascota = Mascota(
                    id = ruta.mascotaId ?: 0, nombre = nombre, especie = especie, raza = raza,
                    fechaNacimiento = fechaNacimiento, sexo = sexo, pesoKg = pesoKg.toDoubleOrNull() ?:
                )
                if (ruta.mascotaId == null) repository.guardar(mascota) else repository.actualizar(masco
                _eventos.send(RegistrarMascotaEvento.GuardadoExitoso)
            } catch (e: Exception) {
                _eventos.send(RegistrarMascotaEvento.Error(e.message ?: "No se pudo guardar"))
            }
        }
    }
}
```

`RegistrarMascotaRoute`/`RegistrarMascotaScreen` siguen el mismo patrón que `RegistrarCuidadoScreen` (Route colecta `uiState` + escucha `eventos` con `LaunchedEffect` para volver al guardar). Formulario con `OutlinedTextField` (nombre, raza, fecha, peso) y `ExposedDropdownMenuBox` para especie/sexo. Wireframe "Pantalla 4 — Registrar/editar mascota".

Persona C — `DetalleMascotaScreen.kt`/`DetalleMascotaViewModel.kt` *(ampliada) +* `RegistrarCuidadoScreen.kt`/`RegistrarCuidadoViewModel.kt` *(sin cambios)*

`DetalleMascotaViewModel` gana una acción de eliminar y un canal de eventos puntuales (antes no tenía ninguno):

```
sealed interface DetalleEvento {
    data object MascotaEliminada : DetalleEvento
    data class Error(val mensaje: String) : DetalleEvento
}

@HiltViewModel
class DetalleMascotaViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val mascotaRepository: MascotaRepository,
    cuidadoRepository: CuidadoRepository
) : ViewModel() {
    private val ruta = savedStateHandle.toRoute<PetCareRoute.Detalle>()

    private val _eventos = Channel<DetalleEvento>()
```

```
    val eventos = _eventos.receiveAsFlow()

    val uiState: StateFlow<DetalleUiState> =
        combine(
            mascotaRepository.obtenerMascotaPorId(ruta.mascotaId),
            cuidadoRepository.obtenerCuidadosDeMascota(ruta.mascotaId)
        ) { mascota, cuidados ->
            if (mascota != null) DetalleUiState.Success(mascota, cuidados)
            else DetalleUiState.Error("Mascota no encontrada")
        }
            .catch { emit(DetalleUiState.Error(it.message ?: "Error al cargar")) }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), DetalleUiState.Loading)

    fun eliminarMascota() {
        viewModelScope.launch {
            val estado = uiState.value
            if (estado is DetalleUiState.Success) {
                try {
                    mascotaRepository.eliminar(estado.mascota)
                    _eventos.send(DetalleEvento.MascotaEliminada)
                } catch (e: Exception) {
                    _eventos.send(DetalleEvento.Error(e.message ?: "No se pudo eliminar"))
                }
            }
        }
    }
}
```

`DetalleMascotaScreen` agrega botones "Editar mascota" (→ `onEditarMascotaClick`) y "Eliminar mascota" (con un `AlertDialog` de confirmación antes de llamar a `viewModel.eliminarMascota()`). `DetalleMascotaRoute` escucha `eventos` con `LaunchedEffect`: si llega `MascotaEliminada`, llama a `onMascotaEliminada()` (el contenedor hace `popBackStack()`).

`RegistrarCuidadoScreen.kt`/`RegistrarCuidadoViewModel.kt` quedan igual que en la versión anterior de este documento.

Persona D — `ConfiguracionScreen.kt`/`ConfiguracionViewModel.kt` (nueva) + `PetCareApp.kt` (NavHost + bottom bar, ampliado)

```
sealed interface ConfiguracionUiState {
    data object Loading : ConfiguracionUiState
    data class Success(val preferencias: UserPreferences) : ConfiguracionUiState
}

@HiltViewModel
class ConfiguracionViewModel @Inject constructor(
    private val repository: PreferenciasRepository
) : ViewModel() {
    val uiState: StateFlow<ConfiguracionUiState> = repository.preferencias
        .map { ConfiguracionUiState.Success(it) as ConfiguracionUiState }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), ConfiguracionUiState.Loading)

    fun cambiarTema(tema: Tema) = viewModelScope.launch { repository.actualizarTema(tema) }
    fun cambiarOrdenMascotas(orden: OrdenMascotas) = viewModelScope.launch { repository.actualizarOrdenM
    fun cambiarOrdenHistorial(orden: OrdenHistorial) = viewModelScope.launch { repository.actualizarOrde
}
```

`ConfiguracionRoute`/`ConfiguracionScreen` dibujan 3 controles (tema, orden de mascotas, orden del historial) según wireframe "Pantalla 9 — Configuración", cada uno llamando a su función del ViewModel al cambiar.

```
// PetCareApp.kt — ahora con bottom bar condicional + 7 rutas
@Composable
fun PetCareApp(navController: NavHostController = rememberNavController()) {
    val backStackEntry by navController.currentBackStackEntryAsState()
    val rutaActual = backStackEntry?.destination
    val mostrarBottomBar = rutaActual?.hasRoute<PetCareRoute.Home>() == true ||
        rutaActual?.hasRoute<PetCareRoute.Mascotas>() == true ||
        rutaActual?.hasRoute<PetCareRoute.Configuracion>() == true

    Scaffold(
        bottomBar = { if (mostrarBottomBar) PetCareBottomBar(navController, rutaActual) }
    ) { padding ->
        NavHost(
            navController = navController,
            startDestination = PetCareRoute.Splash,
            modifier = Modifier.padding(padding)
        ) {
            composable<PetCareRoute.Splash> {
                SplashRoute(onTimeout = {
                    navController.navigate(PetCareRoute.Home) {
                        popUpTo(PetCareRoute.Splash) { inclusive = true }
                    }
                })
            }
            composable<PetCareRoute.Home> {
                HomeRoute(onVerMascotasClick = { navController.navigate(PetCareRoute.Mascotas) })
            }
            composable<PetCareRoute.Mascotas> {
                MascotasRoute(
                    onMascotaClick = { id -> navController.navigate(PetCareRoute.Detalle(mascotaId = id)
                    onAgregarMascotaClick = { navController.navigate(PetCareRoute.RegistrarMascota()) }
                )
            }
            composable<PetCareRoute.Configuracion> {
                ConfiguracionRoute()
            }
            composable<PetCareRoute.Detalle> { backStackEntry ->
                val ruta = backStackEntry.toRoute<PetCareRoute.Detalle>()
                DetalleMascotaRoute(
                    onEditarMascotaClick = { navController.navigate(PetCareRoute.RegistrarMascota(mascot
                    onMascotaEliminada = { navController.popBackStack() },
                    onRegistrarCuidadoClick = { navController.navigate(PetCareRoute.RegistrarCuidado(mas
                    onCuidadoClick = { cuidadoId ->
                        navController.navigate(PetCareRoute.RegistrarCuidado(mascotaId = ruta.mascotaId,
                    },
                    onBackClick = { navController.popBackStack() }
                )
            }
            composable<PetCareRoute.RegistrarMascota> {
                RegistrarMascotaRoute(onListo = { navController.popBackStack() })
            }
            composable<PetCareRoute.RegistrarCuidado> {
                RegistrarCuidadoRoute(onListo = { navController.popBackStack() })
            }
        }
    }
}

@Composable
private fun PetCareBottomBar(navController: NavHostController, rutaActual: NavDestination?) {
    NavigationBar {
```

```
            NavigationBarItem(
                selected = rutaActual?.hasRoute<PetCareRoute.Home>() == true,
                onClick = {
                    navController.navigate(PetCareRoute.Home) {
                        popUpTo(PetCareRoute.Home) { saveState = true }
                        launchSingleTop = true
                        restoreState = true
                    }
                },
                icon = { Icon(Icons.Default.Home, contentDescription = null) },
                label = { Text("Inicio") }
            )
            NavigationBarItem(
                selected = rutaActual?.hasRoute<PetCareRoute.Mascotas>() == true,
                onClick = {
                    navController.navigate(PetCareRoute.Mascotas) {
                        popUpTo(PetCareRoute.Home) { saveState = true }
                        launchSingleTop = true
                        restoreState = true
                    }
                },
                icon = { Icon(Icons.Default.Pets, contentDescription = null) },
                label = { Text("Mascotas") }
            )
            NavigationBarItem(
                selected = rutaActual?.hasRoute<PetCareRoute.Configuracion>() == true,
                onClick = {
                    navController.navigate(PetCareRoute.Configuracion) {
                        popUpTo(PetCareRoute.Home) { saveState = true }
                        launchSingleTop = true
                        restoreState = true
                    }
                },
                icon = { Icon(Icons.Default.Settings, contentDescription = null) },
                label = { Text("Ajustes") }
            )
        }
    }
```

Esta es la implementación estándar recomendada de Google para combinar `NavigationBar` + Navigation Compose (`popUpTo`/`launchSingleTop`/`restoreState` evitan apilar copias de la misma pantalla al tocar varias veces la misma pestaña).

Paso 2 — Integración y verificación

1. Cada persona sube su(s) archivo(s) a su propia rama y abre PR contra `main`.
2. Se integra en orden: `data/` + `di/` + `PetCareApplication.kt` + `PetCareRoute.kt`, luego las 7 pantallas, luego `PetCareApp.kt` + `MainActivity.kt` + `AndroidManifest.xml`.
3. Compilar y correr. Probar a mano **el flujo completo**:

- Splash → Home (con el conteo real de mascotas).
- Bottom bar: Inicio ↔ Mascotas ↔ Ajustes, sin duplicar pantallas en la pila.
- Mascotas → "+" → Registrar mascota → guardar → aparece en la lista.
- Tocar una mascota → Detalle → Editar → cambiar un dato → guardar → se refleja.

- Detalle → Eliminar mascota (con confirmación) → vuelve a Mascotas y ya no aparece (ni sus cuidados).
- Detalle → Registrar cuidado → guardar/editar/eliminar un cuidado, igual que antes.
- Configuración: cambiar tema/orden y volver a abrir la app — debe recordarlo (DataStore persiste).
- Botón físico de "atrás" en cada pantalla.
- **Offline-first**: cerrar la app, modo avión, reabrir — todo debe seguir ahí.

1 Revisar que ningún `XScreen` reciba el ViewModel directamente, y que nada se instancie fuera de Hilt.

Recordatorio: presentación de cierre (elevator pitch, 5 minutos)

- **1 min** — nombre de la app y el problema que resuelve.
- **2 min** — demo en vivo: crear una mascota, registrar/editar/eliminar un cuidado, cambiar una preferencia en Configuración, navegación con bottom bar + botón físico de atrás, y la prueba offline (cerrar app + modo avión + reabrir).
- **2 min** — arquitectura: `Route`/`Screen` sin pasar el ViewModel, `StateFlow`/`Flow` desde Room/DataStore hasta la UI, y cómo Hilt inyecta repositorios y ViewModels.
- Los 4 integrantes deben poder responder preguntas de cualquier capa.

Fuera de alcance (mejoras opcionales, no requisitos)

- Aplicar visualmente el tema y el orden de las listas en el resto de la app (Configuración ya guarda/lee correctamente con DataStore, que es lo que pide RF10).
- Pestaña global "Cuidados" en la bottom bar (no especificada en ningún requisito funcional).
- Casos de uso (`UseCases`) — extra opcional (+0.3 pts), no hay lógica de negocio compleja todavía.
- Ícono adaptativo vectorizado (+0.3 pts) y `WindowSizeClass` (+0.4 pts) — extras opcionales.
- Modelos de IA on-device — mencionados como posibilidad, no aplica al alcance de este equipo.